Intelligent Agent-Based System for Automated Classification and Prioritization of Customer Support Tickets

Dataset Link:https://www.kaggle.com/datasets/suraj520/customer-support-ticket-dataset/data

Student Name:

Student ID:

Table of Contents

1.0 Application and Workflow Selection 2

1.1 Introduction and Motivation of the Selected Application 4

1.2 Explanation of the Selected Workflow 4

1.3 Objectives of the Selected Intelligent System 5

1.4 Impact of the Selected Project if Developed 5

2.0 Selection of AI Technologies 6

2.1 Identification of Similar Systems 6

2.2 Innovative and Original Approaches in System Design 6

2.3 Justification for Choosing AI Methods and Technologies 6

3.0 Simulations and Development Setup 7

3.1 Simulation Environments and Setup Parameters 7

3.2 Task Distribution, Functionality, and Datasets 8

3.3 Encoding and Data Processing Prior to Simulation 9

4.0 Use Case Explanation with Evidence 10

4.1 Step-by-Step Workflow Execution with Evidence 10

4.2 Developed User Interface and External Interaction 16

5.0 Conclusions 17

Reference 18

List of Figures:

Figure 1 Workflow Diagram 5

Figure 2 Simulation Environments and Setup Parameters 8

Figure 3 Dataset overview 9

Figure 4 Encoding and Data Processing 10

Figure 5 Data Processing Agent 11

Figure 6 Feature Engineering Agent 12

Figure 7 Model Training Agent 13

Figure 8 Evaluation Agent 15
Figure 9 Model Comparison and Selection Agent 16

1.0 Application and Workflow Selection

1.1 Introduction and Motivation of the Selected Application

The customer service departments prove instrumental in ensuring the customer satisfaction and brand image. Thousands of customer support tickets are received by modern organizations via different communication channels every day, including email, chat, and web portals. The manual classification and prioritization of these tickets are inefficient and prone to errors since such tickets usually vary in urgency, type, and complexity. The more the tickets are sold, the less opportunity do human agents have to provide an immediate response and find a solution to a problem, which contributes to the decreased customer satisfaction(Yi and Liu, 2020).

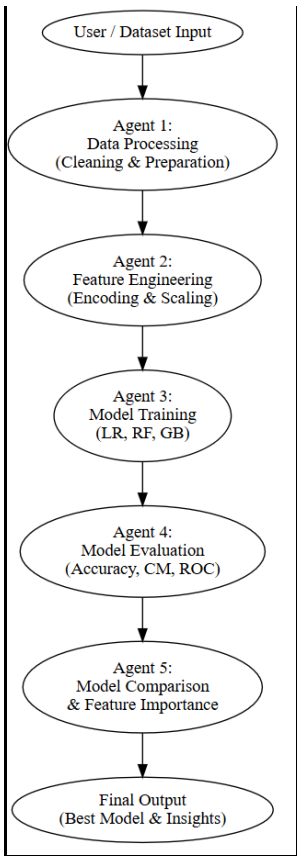
This project will be driven by the aim of curbing such problems by creating an intelligent system that can automatically analyze and classify customer support tickets using historical data. Through an agentic AI framework that utilizes machine learning to predict ticket status, the system will assist the customer service teams to work more efficiently by having high accuracy and sameness in prediction(Sun, Liu and Gao, 2023). The automation is associated with quicker decision-making, resource allocation optimization, and enhanced quality of the services, which makes it a highly applicable application in the real customer service setting.

1.2 Explanation of the Selected Workflow

[redacted]

[redacted]

[redacted]



[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

```python
# Install Kaggle
!pip install -q kaggle

KAGGLE_USERNAME = "your_kaggle_username"
KAGGLE_KEY = "your_kaggle_api_key"


import os

# Set environment variables
os.environ["KAGGLE_USERNAME"] = KAGGLE_USERNAME
os.environ["KAGGLE_KEY"] = KAGGLE_KEY

# Create directory for dataset
os.makedirs("customer_support_ticket_dataset", exist_ok=True)

# Download the dataset
!kaggle datasets download -d suraj520/customer-support-ticket-dataset -p customer_support_ticket_dataset

# Unzip the dataset
!unzip -o customer_support_ticket_dataset/customer-support-ticket-dataset.zip -d customer_support_ticket_dataset

# List downloaded files
os.listdir("customer_support_ticket_dataset")
```

```
Dataset URL: https://www.kaggle.com/datasets/suraj520/customer-support-ticket-dataset
License(s): CC0-1.0
Downloading customer-support-ticket-dataset.zip to customer_support_ticket_dataset
  0% 0.00/828k [00:00<?, ?B/s]
100% 828k/828k [00:00<00:00, 889MB/s]
Archive:  customer_support_ticket_dataset/customer-support-ticket-dataset.zip
  inflating: customer_support_ticket_dataset/customer_support_tickets.csv
```

```python
import zipfile
import os
import pandas as pd
```

```python
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

```python
from sklearn.preprocessing import LabelEncoder, StandardScaler
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
from sklearn.model_selection import train_test_split
```

[redacted]

[redacted]

[redacted]

[redacted]

```python
print("Dataset extracted successfully!")

# CSV file path (already known from output)
csv_file = os.path.join(
    extract_path,
    "customer_support_tickets.csv"
)

# Load into DataFrame
df = pd.read_csv(csv_file)

# Display first few rows
df.head()
```

```
Dataset extracted successfully!
```

	Ticket ID	Customer Name	Customer Email	Customer Age	Customer Gender	Product Purchased	Date of Purchase	Ticket Type	Ticket Subject	Ticket Description
0	1	Marisa Obrien	carrollallison@example.com	32	Other	GoPro Hero	2021-03-22	Technical issue	Product setup	I'm having an issue with the {product_purchase...
1	2	Jessica Rios	clarkeashley@example.com	42	Female	LG Smart TV	2021-05-22	Technical issue	Peripheral compatibility	I'm having an issue with the {product_purchase...
2	3	Christopher Robbins	gonzalestracy@example.com	48	Other	Dell XPS	2020-07-14	Technical issue	Network problem	I'm facing a problem with my {product_purchase...

[redacted]

[redacted]

[redacted]

[redacted]

```python
import matplotlib.pyplot as plt
import seaborn as sns

# Drop non-informative / leakage columns
drop_cols = [
    "Ticket ID", "Customer Name", "Customer Email",
    "Date of Purchase", "Ticket Subject", "Ticket Description",
    "Resolution", "First Response Time", "Time to Resolution"
]

df_model = df.drop(columns=drop_cols)

# Separate target
target = "Ticket Status"
y = df_model[target]
X = df_model.drop(columns=[target])

# Convert categorical columns to string
cat_cols = X.select_dtypes(include="object").columns
num_cols = X.select_dtypes(include=["int64", "float64"]).columns

for col in cat_cols:
    X[col] = X[col].astype(str)

# Encode categorical features
le = LabelEncoder()
for col in cat_cols:
    X[col] = le.fit_transform(X[col])

# Scale numeric features
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

[redacted]

[redacted]

[redacted]

[redacted]

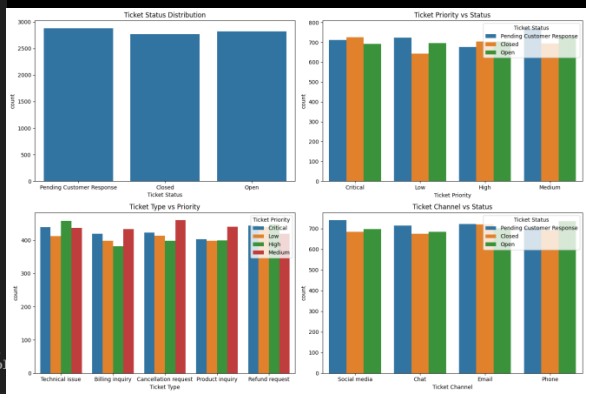
[redacted]

[redacted]

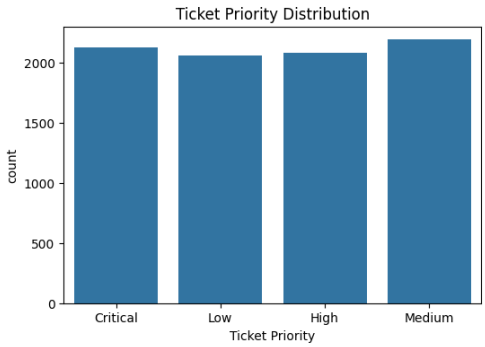
[redacted]

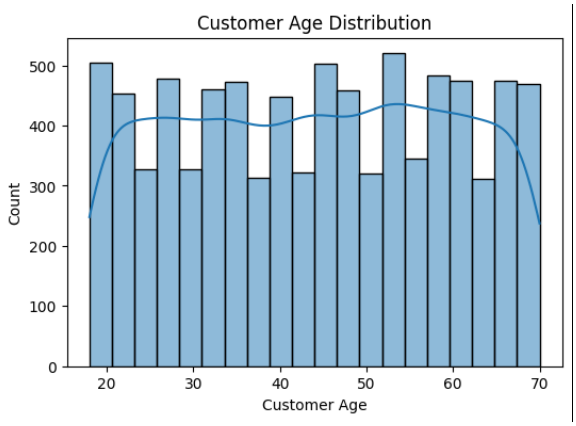
[redacted]

```
(8469, 17)
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 8469 entries, 0 to 8468
Data columns (total 17 columns):
 #   Column                        Non-Null Count  Dtype
---  ------                        --------------  -----
 0   Ticket ID                     8469 non-null   int64
 1   Customer Name                 8469 non-null   object
 2   Customer Email                8469 non-null   object
 3   Customer Age                  8469 non-null   int64
 4   Customer Gender               8469 non-null   object
 5   Product Purchased             8469 non-null   object
 6   Date of Purchase              8469 non-null   object
 7   Ticket Type                   8469 non-null   object
 8   Ticket Subject                8469 non-null   object
 9   Ticket Description            8469 non-null   object
 10  Ticket Status                 8469 non-null   object
 11  Resolution                    2769 non-null   object
 12  Ticket Priority               8469 non-null   object
 13  Ticket Channel                8469 non-null   object
 14  First Response Time           5650 non-null   object
 15  Time to Resolution            2769 non-null   object
 16  Customer Satisfaction Rating  2769 non-null   float64
dtypes: float64(1), int64(2), object(14)
memory usage: 1.1+ MB
...
First Response Time             2819
Time to Resolution              5700
Customer Satisfaction Rating    5700
dtype: int64
Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...
/tmp/ipython-input-19748315.py:16: FutureWarning: Setting an item of incompatib
  df.fillna("Unknown", inplace=True)
```



[redacted]





[redacted]

[redacted]

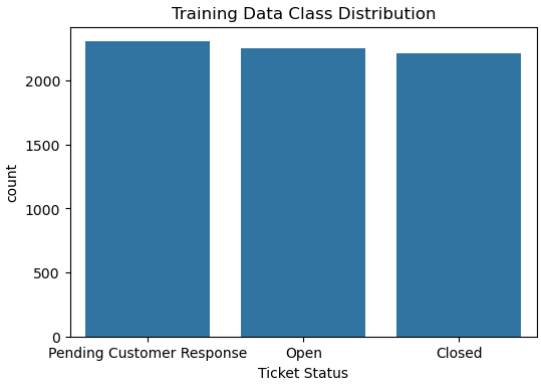
[redacted]

[redacted]

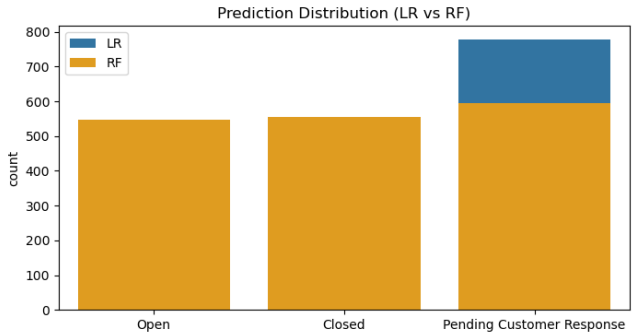
[redacted]

[redacted]

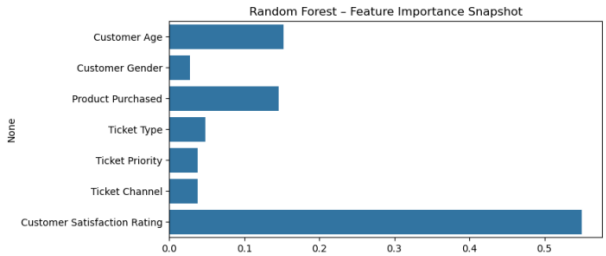
[redacted]



[redacted]



[redacted]




```python
# Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

# Model 1: Logistic Regression
lr_model = LogisticRegression(max_iter=1000)
lr_model.fit(X_train, y_train)

# Model 2: Random Forest
rf_model = RandomForestClassifier(
    n_estimators=200,
    max_depth=15,
    random_state=42
)
rf_model.fit(X_train, y_train)

# Model 3: Gradient Boosting
gb_model = GradientBoostingClassifier(random_state=42)
gb_model.fit(X_train, y_train)
```

```
GradientBoostingClassifier
GradientBoostingClassifier(random_state=42)
```

[redacted]

[redacted]

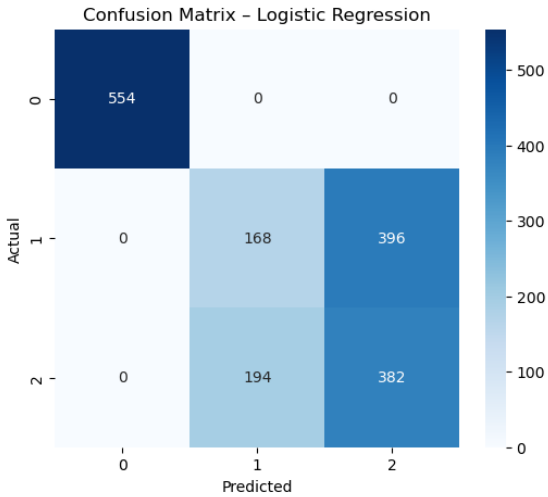
[redacted]

[redacted]

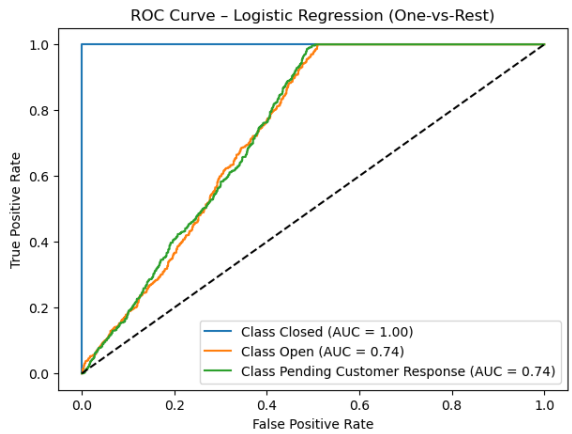
[redacted]

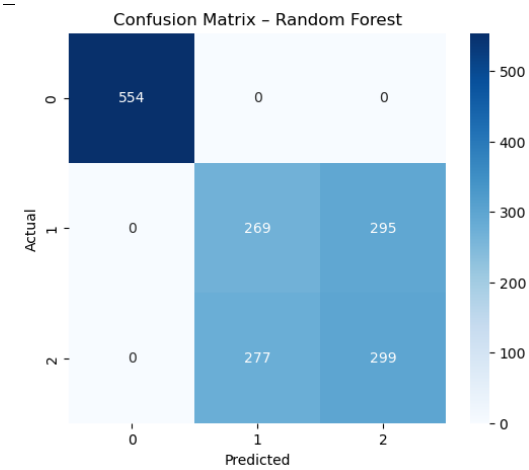
[redacted]

[redacted]

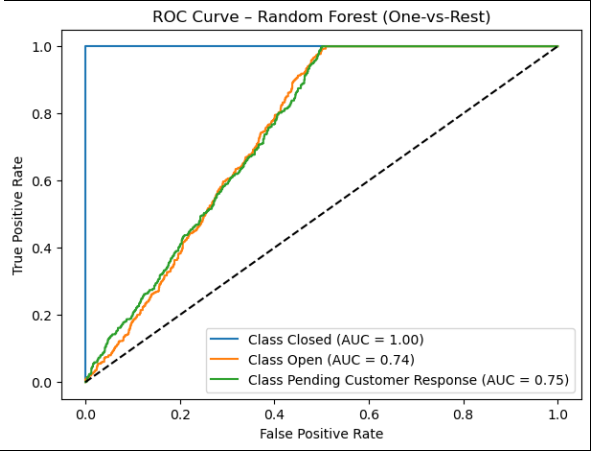


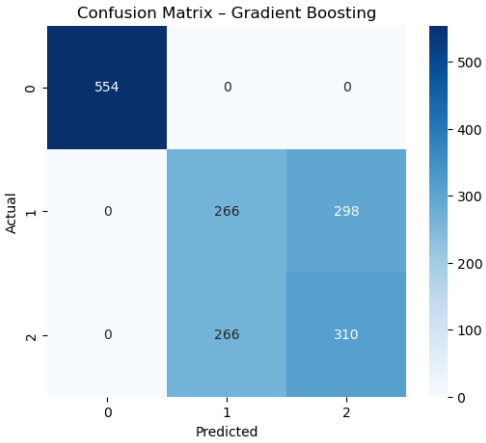
[redacted]



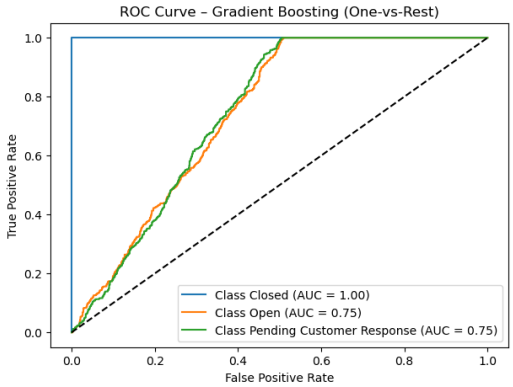


[redacted]





[redacted]

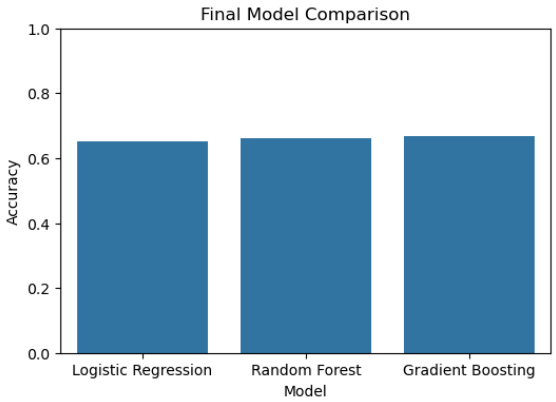


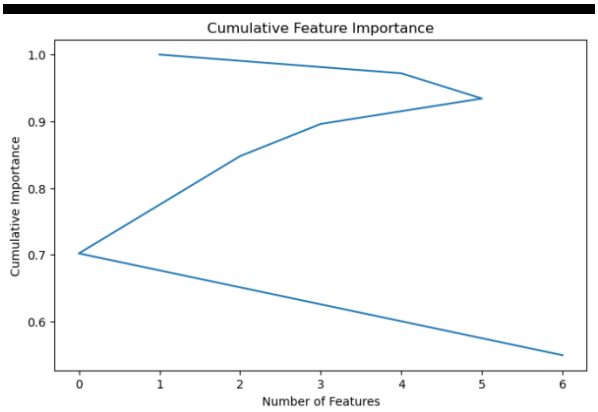

```
Logistic Regression
                          precision    recall  f1-score   support

                  Closed       1.00      1.00      1.00       554
                    Open       0.46      0.30      0.36       564
Pending Customer Response      0.49      0.66      0.56       576

                accuracy                           0.65      1694
               macro avg       0.65      0.65      0.64      1694
            weighted avg       0.65      0.65      0.64      1694


Random Forest
                          precision    recall  f1-score   support

                  Closed       1.00      1.00      1.00       554
                    Open       0.49      0.48      0.48       564
Pending Customer Response      0.50      0.52      0.51       576

                accuracy                           0.66      1694
               macro avg       0.67      0.67      0.67      1694
            weighted avg       0.66      0.66      0.66      1694

...
                accuracy                           0.67      1694
               macro avg       0.67      0.67      0.67      1694
            weighted avg       0.67      0.67      0.67      1694
```

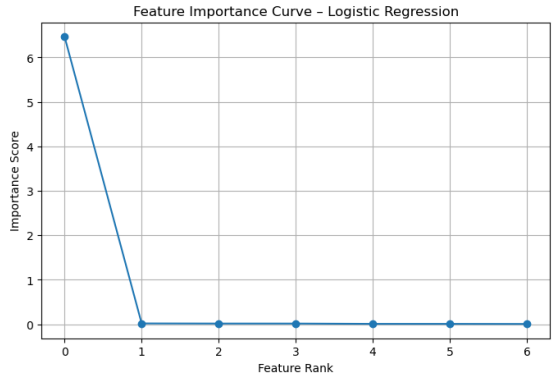
[redacted]

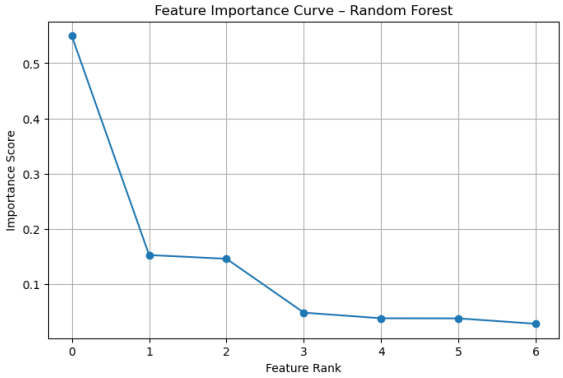
[redacted]

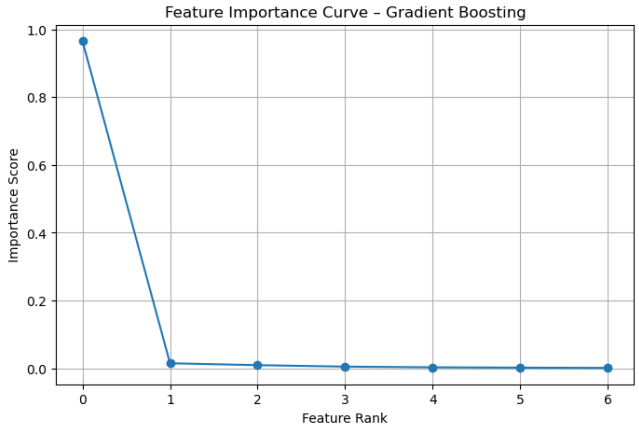
[redacted]











[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]